

Evaluation of Model-Free Reinforcement Learning Algorithms

Metasebiya Mulugeta GSE/2623/17

June 26, 2026

Abstract

In this report, two core classes of model-free Reinforcement Learning techniques are analyzed and evaluated. In the first exercise, First-Visit Monte Carlo (MC) Policy Evaluation is utilized to approximate the state-value function $V(s)$ for a fixed threshold policy in the game of Blackjack. In the second exercise, the on-policy Temporal Difference (TD) control algorithm, SARSA, is implemented to determine an exploration-aware policy within a gridworld Cliff Walking environment. The theoretical frameworks, implementation paradigms, and empirical convergence properties are systematically reported using experimental outputs and graphical illustrations.

1 Exercise 1: Monte Carlo Blackjack Policy Evaluation

1.1 Theoretical Background

Monte Carlo (MC) methods are applied to estimate value functions and discover optimal policies in episodic, model-free environments where transition probabilities $P(s' | s, a)$ are unknown. Unlike Dynamic Programming, which relies on full bootstrapping from a transition model, MC approaches learn directly from sampled trajectories of complete episodes. For policy evaluation, the objective is to approximate the state-value function $V^\pi(s)$, representing the expected return starting from state s and following policy π thereafter. The total discounted return following time step t is formulated as:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \quad (1)$$

where $\gamma \in [0, 1]$ represents the discount factor and T is the terminal time step. Under the First-Visit MC method, the return G_t is recorded and averaged only upon the absolute first instance a unique state s is visited within a given episode. The value function is incrementally updated using the sample mean approach:

$$N(s) \leftarrow N(s) + 1 \quad (2)$$

$$V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)] \quad (3)$$

where $N(s)$ is a counter tracking the historical count of first visits to state s .

1.2 Environment and Policy Setup

The standard OpenAI Gym/Gymnasium **Blackjack-v1** environment was chosen for simulation. The state space is characterized as a three-dimensional tuple:

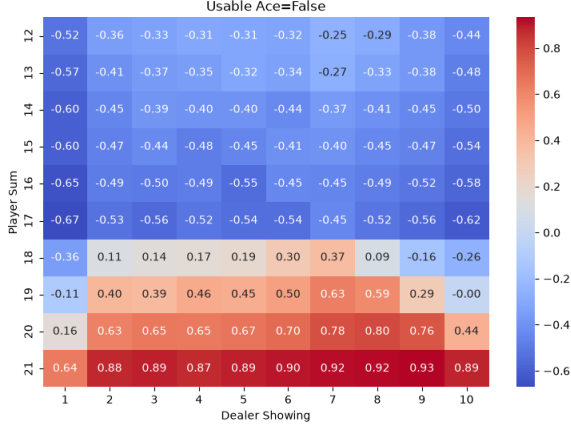
$$s = (\text{Player's Current Sum}, \text{Dealer's Visible Card}, \text{Usable Ace Flag}) \quad (4)$$

where the player's sum ranges from 12 to 21, the dealer's visible card spans 1 to 10 (where 1 represents an Ace), and the usable ace flag is a boolean denoting whether an Ace can be counted as 11 without busting. A fixed threshold policy π was evaluated. The action choice $a \in \{\text{Stick}, \text{Hit}\}$ is strictly governed by the rule:

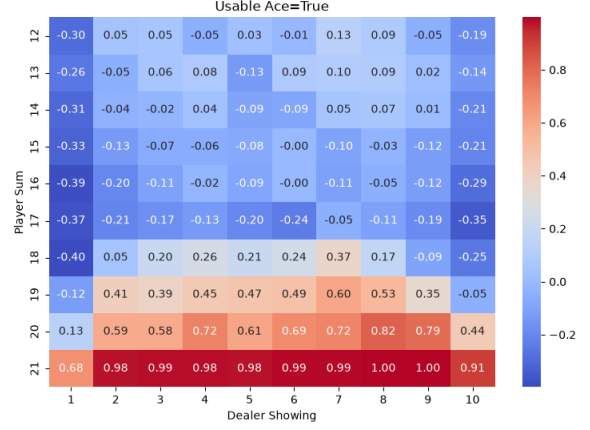
$$\pi(s) = \begin{cases} \text{Stick} & \text{if Player's Sum} \geq 18 \\ \text{Hit} & \text{otherwise} \end{cases} \quad (5)$$

1.3 Experimental Results and Visualizations

The first-visit MC algorithm was executed over multiple independent trajectories to reconstruct the value function surface. The empirical results were partitioned based on the presence of a usable ace.



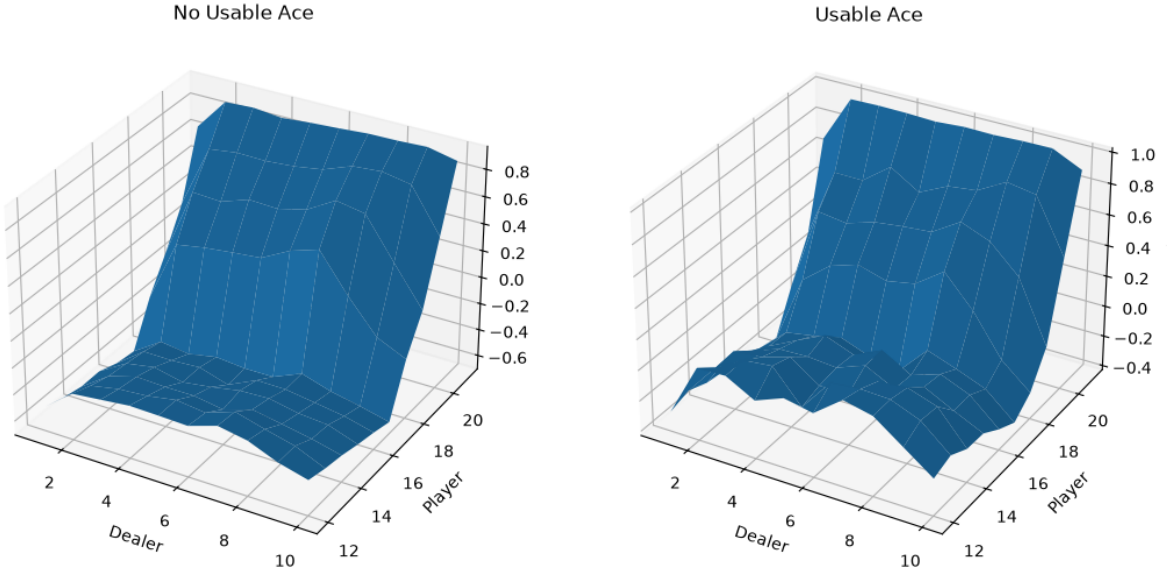
(a) Heatmap without Usable Ace



(b) Heatmap with Usable Ace

Figure 1: Heatmap distributions of state-value function $V(s)$ across dealer showing and player sum coordinates.

As displayed in the flat heatmaps (Figure 1), state-value distributions exhibit structural shifts when an Ace can be flexibly downgraded from 11 to 1. The presence of a usable ace introduces a protective layer against immediate termination (busting), leading to universally higher expected returns across mid-tier player scores.



(a) No Usable Ace Surface 3D

(b) Usable Ace Surface 3D

Figure 2: 3D perspective projections of the learned state-value function $V(s)$.

The 3D visualizations (Figure 2) confirm a predictable “step-up” phenomenon. High value estimates dominate the area where the player’s sum is 20 or 21. Conversely, states where the player’s score is low (e.g., 12–15) produce low value heights due to a severe vulnerability to busting under the aggressive hitting mandatory below 18.

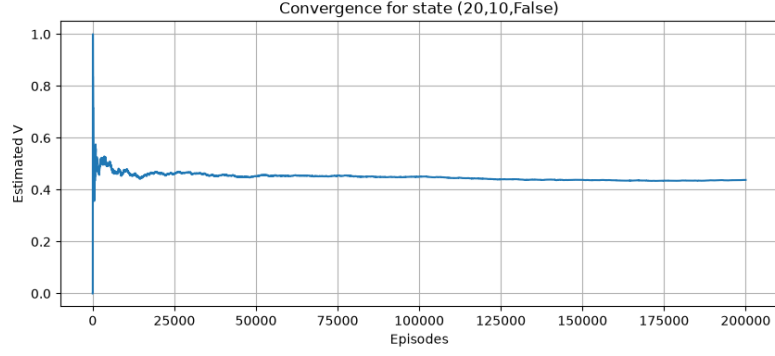


Figure 3: Value estimate stabilization plotted against the total number of sampled episodes.

The convergence curve (Figure 3) tracks changes in $V(s)$ for designated baseline states. Early high-variance oscillations flatten as sample counts grow, satisfying the Law of Large Numbers.

To provide a more concrete view of the learned value estimates, Table 1 reports a subset of representative state values for player sums 18 and 19 under the fixed threshold policy.

Table 1: Representative Monte Carlo state-value estimates for selected blackjack states.

Player Sum	Dealer	Value
18	1	-0.355964
18	2	0.106984
18	3	0.143794
18	4	0.167656
18	5	0.188143
18	6	0.301145
18	7	0.370690
18	8	0.088309
18	9	-0.156530
18	10	-0.256311
19	1	-0.112288
19	2	0.400521
19	3	0.387646
19	4	0.460439
19	5	0.450469
19	6	0.502318
19	7	0.627420
19	8	0.589446
19	9	0.289744
19	10	-0.001521

2 Exercise 2: Temporal Difference - Cliff Walking with SARSA

2.1 Theoretical Background

Temporal Difference (TD) learning represents a blend of Monte Carlo and Dynamic Programming concepts. Like MC, TD methods learn directly from experienced transitions without requiring an environmental model. However, unlike MC, TD methods update value estimates using a technique called bootstrapping, where subsequent predictions adjust preceding ones before a final episode termination is reached.

SARSA (State-Action-Reward-State-Action) is an on-policy TD control method. It learns the action-value function $Q(s, a)$ associated with the policy currently being executed. The foundational update formulation is driven by the rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)] \quad (6)$$

where $\alpha \in (0, 1]$ represents the learning parameter, γ is the discount rate, and the TD target is explicitly composed using the actual next action a' selected under the current active exploration framework.

2.2 Environment and Policy Setup

The problem was evaluated within Gymnasium's `CliffWalking-v1` gridworld. The grid is organized in a 4×12 matrix layout. The states are handled as discrete integer values spanning 0 to 47. The dynamics are mapped as follows:

- **Start State:** State 36 (Bottom-left corner).
- **Goal State:** State 47 (Bottom-right corner).
- **Cliff Zone:** States 37 to 46 inclusive (Bottom row between start and goal).
- **Reward Framework:** A step penalty of -1 is allocated across safe movements. Stepping into a cliff index drops a severe negative penalty of -100 and immediately forces a reset back to Start State 36.

Action selection is handled via an ϵ -greedy mechanism, where with a small probability ϵ , a uniform random exploratory action is executed; otherwise, the greedy exploitation criteria $\operatorname{argmax}_a Q(s, a)$ is prioritized.

2.3 Experimental Results and Outputs

The training algorithm was configured with parameters $\alpha = 0.5$, $\gamma = 0.99$, and $\epsilon = 0.1$, running for a total duration of 500 episodes. The training completion details and final extracted action-value array lists for the leftmost column vector are shown below:

```
Number of states: 48
Number of actions: 4
Training completed.
State 36: [ -18.81798171 -118.13025125 -20.68418017 -20.20862356]
State 24: [-17.96545337 -23.89683443 -20.67930286 -19.8110278 ]
State 12: [-19.41169904 -16.76377904 -20.11662608 -19.25068623]
State 0: [-19.07543663 -15.93068573 -19.54361816 -19.20212551]
```

Listing 1: Console output showing state-action value metrics after 500 training episodes.

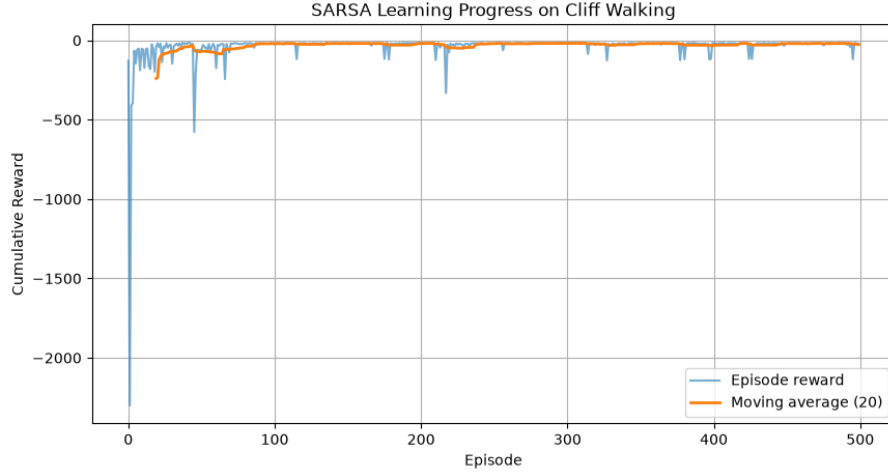


Figure 4: Episode cumulative rewards and a 20-episode moving average curve.

The progression plot (Figure 4) demonstrates clear optimization behavior. Early steps are dominated by massive negative feedback caused by frequent cliff falls. As the state-action matrix updates, cumulative reward parameters trend upward, stabilizing near the maximum bounds permitted within an exploration-penalized framework.

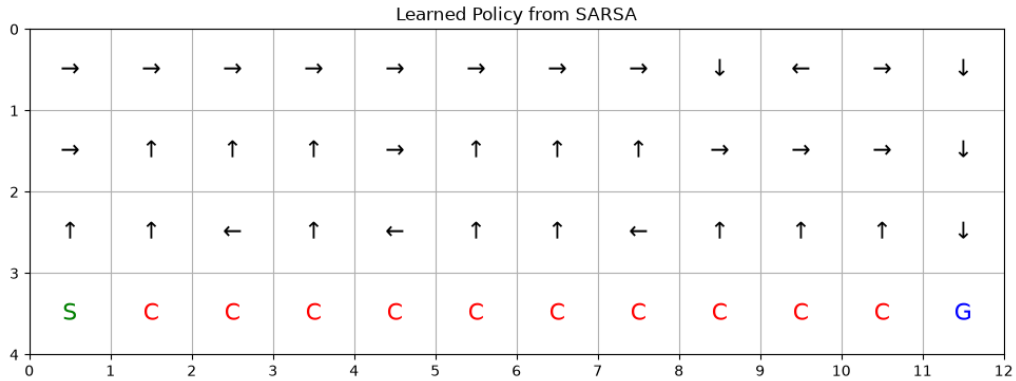


Figure 5: Visual orientation layout of the policy grid mapped out by the SARSA agent.

The synthesized navigation direction layout is detailed in Figure 5. The on-policy update mechanism forces the agent to discover a structurally robust “safe path.” Because exploration steps are actively evaluated inside the updates, traveling on the raw boundary line adjacent to the cliff matrix is deemed sub-optimal due to the high risk of random exploratory actions causing a cliff reset. The agent is observed prioritizing a path that loops upward, providing a safety buffer to mitigate the reward penalties of accidental ϵ -driven exploration.

3 Conclusion

Model-free validation has been presented across distinct environments. First-Visit Monte Carlo tracking provided an unbiased value mapping for episodic, non-bootstrapping card tasks. Temporal Difference learning via SARSA showed rapid adaptation in step-by-step grid assignments, validating how on-policy adjustments naturally account for current exploration risks to discover balanced, safe trajectories.